

Lernziele:

- Vertiefung: Listen und Umgang mit Strings

Vorbereitung: Compiler aktualisieren

1. Wechselt in das Verzeichnis **abc-llvm** mit dem Source-Code für den Compiler.
2. Führt das Kommando **git pull** aus.
3. Führt das Kommando **make** aus.
4. Führt das Kommando **sudo make install** aus.

Zielsetzung

In einem Compiler werden Strings für Variablennamen, Literale und Schlüsselwörter verwendet. In den Datenstrukturen (z.B. in den Knoten eines Expression-Trees oder den Symbol-Tabellen) eines Compilers müssen deshalb sehr viele Strings gespeichert werden. Außerdem müssen Strings sehr oft miteinander verglichen werden, um z.B. zu überprüfen, ob eine Variable bereits definiert wurde.

Legt man für jeden String ein neues Array an, dann wird unnötigerweise sehr viel Speicher verschwendet. Denn viele Strings kommen mehr als einmal vor. Vergleicht man zwei Strings stets mit der **strcmp**-Funktion, dann ist dies teuer, da hier jedes Mal mit einer Schleife die einzelnen Zeichen verglichen werden.

In dieser Session implementieren wir eine einfache String-Klasse, um Strings effizient zu speichern und vergleichen zu können. Die Idee ist folgende:

Alle Strings, die im Compiler vorkommen, "entstehen" im Lexer. Wenn der Lexer ein neues Token gefunden hat, soll dieser prüfen, ob der zugehörige String schon einmal vorkam. Um dies zu verwalten, wird eine einfach verkettete Liste verwendet. Ist der String noch nicht in der Liste, dann wird ein neuer Knoten hinzugefügt. Ansonsten wird ein bereits in der Liste verwendeter String wiederverwendet.

Damit der Code für den Lexer übersichtlich bleibt, werden wir ein neues Modul "**ustr**" (für "unique string") implementieren. Dies kann man zunächst völlig unabhängig vom restlichen Projekt implementieren und testen.

Schritt 1 (ustr-Modul)

Im Header **ustr.hdr** wird nur eine Funktion **UStrCreate** deklariert. Als Parameter erwartet diese einen Zeiger auf einen String. Der Rückgabewert ist ebenfalls ein Zeiger auf einen String. Dass beim Rückgabebetyp ein Typ-Alias **UStr** verwendet wird, ist lediglich eine Art von Dokumentation. Hier wird angedeutet, dass der String, den man bekommt, auf eine Datenstruktur zeigt, die keine Duplikate bei Strings zulässt:

```
type UStr: readonly char;  
  
extern fn UStrCreate(s: -> char): -> UStr;
```

Die Verwendung des Moduls wird klarer, wenn man dazu ein einfaches Testprogramm **xtest_ustr.abc** betrachtet:

```

1 @ <stdio.h>
2 @ "ustr.hdr"
3
4 fn main()
5 {
6     local a1: array[] of char = "hallo";
7     local a2: array[] of char = "hello";
8     local a3: array[] of char = "hallo";
9
10    printf("a1 = %p '%s'\n", a1, a1);
11    printf("a2 = %p '%s'\n", a2, a2);
12    printf("a3 = %p '%s'\n", a3, a3);
13
14    local s1: -> UStr = UStrCreate(a1);
15    local s2: -> UStr = UStrCreate(a2);
16    local s3: -> UStr = UStrCreate(a3);
17
18    printf("s1 = %p '%s'\n", s1, s1);
19    printf("s2 = %p '%s'\n", s2, s2);
20    printf("s3 = %p '%s'\n", s3, s3);
21 }

```

In den Zeilen 6 bis 8 werden drei Strings jeweils in Arrays gespeichert. In den Zeilen 10 bis 12 werden diese Strings dann jeweils ausgegeben. Bei der Ausgabe wird nicht nur der String-Inhalt ausgegeben, sondern auch die Adresse, an der der String im Speicher liegt. Hier wird bei der Ausführung auffallen, dass Strings unterschiedliche Adressen haben können, selbst wenn der String-Inhalt gleich ist.

In den Zeilen 14 bis 16 werden die Strings mit **UStrCreate** in die Datenstruktur eingecheckt. Die erhaltenen String-Zeiger werden dann in den Zeilen 18 bis 20 ausgegeben. Hier wird bei der Ausgabe auffallen, dass Strings mit gleichem Inhalt auch an der gleichen Adresse gespeichert sind.

Aufgaben

1. Tippe **ustr.hdr** und **xtest_ustr.abc** jeweils ab. Um Tippfehler auszuschließen, führe make aus. Es sollte nur ein Linker-Fehler erzeugt werden, da **UStrCreate** noch nicht implementiert ist.
2. Für die Implementierung ist ebenfalls schon folgender Code vorgegeben:

```

@ <string.h>
@ <stdlib.h>

@ "ustr.hdr"

struct UStrNode
{
    next: -> UStrNode;
    data: array[1] of char;
};

global list: -> UStrNode;

fn UStrCreate(s: -> char): -> UStr
{
    local n: -> UStrNode = list;
    // TODO 2: Check if string is already in list
    local len: size_t = strlen(s);
    n = malloc(sizeof(*n) + len);
    // TODO 1: Copy string into node and prepend node to list
    return list->data;
}

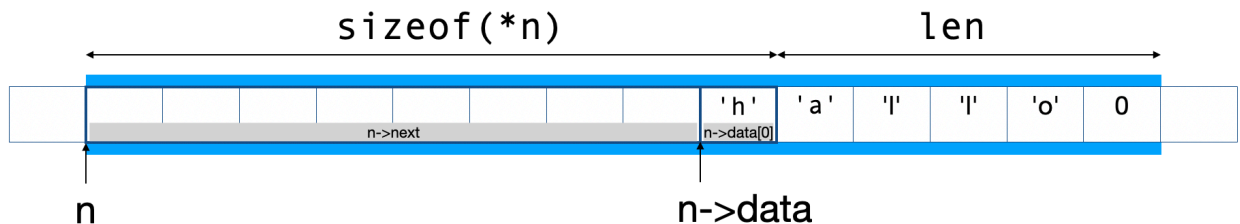
```

Tippt den Code ab. Mit **make** sollte sich dann das Testprogramm in eine ausführbare Datei übersetzen lassen (allerdings wird es noch zu einem Segmentation-Fault kommen).

3. In den weiteren Teilaufgaben soll der Code vervollständigt werden. Dazu solltet ihr euch zunächst mit der Datenstruktur **UStrNode** vertraut machen, die für einen Listenknoten verwendet wird. Dass hier ein Member **next** vorkommt, das auf einen Nachfolgeknoten verweisen kann, ist nicht ungewöhnlich. Erstaunlich ist aber, dass das zweite Member **data** ein Charakter-Array der Dimension 1 ist. Die Größe von **UStrNode** ist deshalb 9 Bytes (wenn ein Zeiger 8 Byte groß ist). Beim Allokieren eines Knotens werden aber mehr Bytes angefordert. Soll in dem Knoten ein String der Länge **len** abgespeichert werden, dann werden zusätzlich **len** Bytes angefordert. Nach

```
n = malloc(sizeof(*n) + len);
```

ist **n** also ein Zeiger auf einen Speicherblock, der wie folgt beschrieben werden kann:



An die Adresse **n->data** kann man also mit **strcpy** problemlos einen Null-terminierten String der Länge **len** kopieren. Natürlich nützt man hier aus, dass in ABC (sowie in C und C++) der Compiler nach dem Prinzip "Trust the programmer" arbeitet.

Implementiert zunächst die mit "TODO 1" gekennzeichnete Stelle. Hier soll der String in den neu allokierten Knoten kopiert werden und der Knoten dann an Anfang der Liste eingehängt werden. Das Testprogramm sollte danach fehlerfrei ausführbar sein. Allerdings werden Strings mit gleichem Inhalt noch in unterschiedlichen Knoten gespeichert.

4. In der letzten Teilaufgabe soll vor dem Einhängen eines neuen Knotens zunächst (mit **strcmp**) überprüft werden, ob der String bereits in einem Knoten **n** der Liste enthalten ist. Ist dies der Fall, wird die Funktion verlassen und **n->data** zurückgegeben.

Schritt 2: Symboltabelle

Für die Verwaltung von globalen Variablen schreiben wir ein neues Modul **genVar**. Im Header sind zwei Funktionen deklariert:

```
@ "ustr.hdr"
```

```
extern fn genAddGlobalVar(name: -> UStr);
extern fn genPrintSymtab();
```

Mit der Funktion **genAddGlobalVar** kann man eine Variable zur Symboltabelle hinzufügen. Ist die Variable bereits enthalten, dann wird dies nicht als Fehler betrachtet, sondern einfach als bereits erledigt aufgefasst. Dass die Funktion mit Strings arbeitet, die zuvor mit **UStrCreate** erzeugt wurden, wird implizit dadurch dokumentiert, dass als Parameter ein Zeiger auf **UStr** erwartet wird. Mit der Funktion **genPrintSymtab** kann Assemblercode für das Datensegment ausgegeben werden für alle Variablen, die zuvor registriert wurden. Im Testprogramm **xtest_genvar.abc** werden beispielsweise die Variablen **x**, **y** und **abc** registriert und dann Assemblercode für die Symboltabelle ausgegeben (dabei wird **x** nur einmal vorkommen, obwohl die Variable zweimal mit

genAddGlobalVar registriert wurde):

```
@ "genvar.hdr"

fn main()
{
    genAddGlobalVar(UStrCreate("x"));
    genAddGlobalVar(UStrCreate("y"));
    genAddGlobalVar(UStrCreate("abc"));
    genAddGlobalVar(UStrCreate("x"));
    genPrintSymtab();
}
```

Aufgaben

1. Tippt **genvar.hdr** und **xtest_genvar.abc** ab. Mit **make** sollte sich wieder alles übersetzen lassen und nur ein Linker-Fehler auftreten.
2. Implementiert in **genvar.abc** die Funktion **genAddGlobalVar**. Hier soll zunächst überprüft werden, ob der Variablenname bereits in der Liste enthalten ist. Natürlich soll jetzt beim Vergleich von Strings nicht **strcmp** verwendet werden, sondern nur die Adressen verglichen werden. Falls der String noch nicht enthalten ist, dann soll ein neuer Knoten mit dem String in die Liste eingehängt werden.

```
@ <stdio.h>
@ <stdlib.h>

@ "genvar.hdr"
@ "ustr.hdr"

struct GlobalNode
{
    next: -> GlobalNode;
    name: -> UStr;
};

global globalList: -> GlobalNode;

fn genAddGlobalVar(name: -> UStr)
{
    // TODO
}

fn genPrintSymtab()
{
    printf("\t.data\n");
    for (local n: -> GlobalNode = globalList; n; n = n->next) {
        printf("\t.align 8\n");
        printf("%s:\n", n->name);
        printf("\t.quad 0\n");
    }
}
```

Schritt 3 Lexer anpassen

In diesem Schritt wird der Lexer angepasst. Im Header ändert sich das Token-Struct zu:

```
struct Token
{
    kind:    TokenKind;
    pos:     struct Pos {
        row, col: unsigned;
    };
    val:     -> UStr;
};
```

Natürlich muss dann auch der Header **ustr.hdr** eingebunden werden. Der Header **string.hdr** und der Typ-Alias **TokenVal** werden dagegen nicht mehr benötigt.

Anpassungen in lexer.abc

Intern werden im Lexer die Zeichen eines neuen Tokens zunächst in einem String-Array gesammelt. Binde deshalb in **lexer.abc** den Header **string.hdr** ein.

Die Zeichen eines Tokens werden in **nextCh** gesammelt. Bisher sieht die Implementierung von **nextCh** in etwa so aus:

```
fn nextCh(updateVal: bool)
{
    if (updateVal) {
        assert(lengthVal < sizeof(token.val) / sizeof(token.val[0]));
        token.val[lengthVal++] = ch;
        token.val[lengthVal] = 0;
    } else {
        lengthVal = 0;
    }
    ch = getchar();
    if (ch == '\n') {
        ++currentPos.row;
        currentPos.col = 0;
    } else {
        ++currentPos.col;
    }
}
```

Bisher war **token.val** ein Array, in dem die Zeichen direkt gesammelt wurden. Da **token.val** jetzt kein Array mehr ist, brauchen wir dazu ein extra Array. Definiere vor **nextCh** das globale Array **val** mit:

```
global val: string;
```

und ersetze in **nextCh** jedes Vorkommen von **token.val** mit **val**.

Anpassung der Funktion getToken

Die Funktion **getToken** ruft intern die Funktion **nextCh** auf, womit die Zeichen eines Tokens in der neuen globalen Array-Variable **val** gesammelt werden. Jetzt sollte man **getToken** noch so anpassen, dass, wenn ein Token gefunden wurde, **val** mit einem Aufruf von **UStrCreate** eingecheckt wird und **token.val** auf den Rückgabewert gesetzt wird. Das würde einige Änderungen erfordern, aber es gibt einen einfachen Trick:

1. Benenne die bisherige Funktion **getToken** um in **getToken_** (oder **getTokenInternal**).

2. Implementiere danach eine Funktion **getToken**, die diese Funktion wie folgt verwendet:

```
fn getToken(): TokenKind
{
    getToken_();
    token.val = UStrCreate(val);
    return token.kind;
}
```

Damit ist **getToken** ein Wrapper für die bisherige **getToken** Funktion, die zusätzlich **token.val** auf den richtigen Wert setzt.

Testen des Lexers

Wenn ihr jetzt `make` aufruft, wird es einige Fehler geben, da auch noch andere Stellen im Projekt angepasst werden müssen. Um den Lexer isoliert testen zu können, könnt ihr aber das Testprogramm für den Lexer mit:

```
abc xtest_lexer.abc lexer.abc ustr.abc
```

direkt übersetzen. Es sollte wie bisher funktionieren. Dass man vielleicht noch `ustr.hdr` einbinden muss, wird euch inzwischen aufgefallen sein.

Schritt 4: Rest des Projektes anpassen

Damit man mit `make` wieder alles übersetzen kann, muss man in weiteren Modulen eigentlich nur noch die bisherige Verwendung von **string** mit **-> UStr** ersetzen und die Einbindung von Headern anpassen (d.h. **ustr.hdr** einbinden und **string.hdr** nicht einbinden). Zum Beispiel sollte man in **expr.hdr** die Deklaration von **Expr** ändern zu:

```
struct Expr
{
    kind:          ExprKind;
    left, right:   -> Expr;
    decimal:       -> UStr;
    identifier:    -> UStr;
};
```

Am einfachsten geht man dabei so vor: `make` ausführen und einen Fehler nach dem anderen beheben.

Schritt 5 Code-Generator anpassen

In **gen.hdr** sollte die Deklaration von **genLoadAddress** geändert werden zu:

```
extern fn genLoadAddress(varName: -> UStr): GenReg;
```

Und in der Implementierung sollte **genLoadAddress** als erstes die Variable mit einem Aufruf von **genAddGlobalVar** in der Symboltabelle registrieren.

Schritt 6 Compiler anpassen

Im letzten Schritt kann der Compiler angepasst werden, so dass dieser am Ende den Assemblercode für das Datensegment ausgibt. Die letzte Anweisung in meinem **xtest_abc.abc** Programm ist zum Beispiel:

```
genPrintSymbtab();
```